Guitar Hero PS3 → ESP32 Mozzi Syntetisaattori

Tarkoitus

Tämä projekti muuttaa käytetyn Guitar Hero 3 PS3 -kitaran itsenäiseksi elektroniseksi soittimeksi. Alkuperäinen kitara on suunniteltu pelkästään pelikäyttöön ja vaatii toimiakseen konsolin, donglen ja pelin — tässä projektissa kaikki tuo korvataan ESP32-mikrokontrollerilla ja Mozzi-syntetisaattorikirjastolla.

Projektin tuloksena syntyy soitin joka toimii täysin itsenäisesti: ei kaapeleita, ei konsolia, ei tietokonetta. Ääni lähtee kitaran sisäisestä kaiuttimesta ja laite toimii ladattavilla akuilla.

Päämäärä

- Uusiokäyttö — vanha pelikitara saa uuden elämän oikeana soittimena
- Itsenäinen laite — kaikki elektroniikka mahtuu kitaran rungon sisään
- Monipuolinen soitin — kaksi soittotilaa, kolme sointusettia, kolme oktaavia, vibrato ja pitch bend
- Helppo soittaa — pop-soinnut yhdellä napilla, pentatoninen asteikko tekee melodioista automaattisesti hyväkuuloisia
- Edullinen rakentaa — komponenttien kokonaishinta noin 10–13€
- Ladattava — USB-C lataus, akku kestää noin 17 tuntia yhdellä latauksella

Tarvittavat taidot

- Juottaminen (perustaso riittää)
- Yleismittarin käyttö jatkuvuusmittaukseen
- Arduino IDE:n peruskäyttö (koodin lataaminen ESP32:een)

Nappien toiminnot

Nappi	Toiminto
Vihreä fret	Nuotti/Sointu 1
Punainen fret	Nuotti/Sointu 2
Keltainen fret	Nuotti/Sointu 3
Sininen fret	Nuotti/Sointu 4

Nappi	Toiminto
Oranssi fret	Nuotti/Sointu 5
Strum alas	Laukaisee äänen
Strum ylös	Laukaisee äänen
Whammy bar	Pitch bend
Select	Vaihda tila: Sointu ↔ Nuotti
Start	Kierrätä sointusetti tai oktaavi
D-pad	Vibrato päälle/pois

Tilat

Tila	Select	Start
Sointutila	Vaihda → Nuottitila	Kierrätä setti (Pop → Rock → Balladi)
Nuottitila	Vaihda → Sointutila	Kierrätä oktaavi (Basso → Normaali → Melodia)

Sointusetit

Setti	Vihreä	Punainen	Keltainen	Sininen	Oranssi
Pop	G	C	D	Am	Em
Rock	E	A	D	G	B
Balladi	C	G	Am	F	Em

Soinnun rakenne

Sointu	Nuotti 1	Nuotti 2	Nuotti 3
G	G3 (196 Hz)	B3 (246.94 Hz)	D4 (293.66 Hz)
C	C3 (130.81 Hz)	E3 (164.81 Hz)	G3 (196 Hz)
D	D3 (146.83 Hz)	F#3 (185 Hz)	A3 (220 Hz)
Am	A3 (220 Hz)	C4 (261.63 Hz)	E4 (329.63 Hz)
Em	E3 (164.81 Hz)	G3 (196 Hz)	B3 (246.94 Hz)

Oktaavit nuottitilassa

Oktaavi	Taajuusalue	Tunnelma
1	Matala (C2–A2)	Basso
2	Keski (C3–A3)	Normaali
3	Korkea (C4–A4)	Melodia

Komponenttilista

Komponentti	Malli	Hinta (AliExpress)
Mikrokontrolleri	ESP32 DevKit	~2€
Vahvistin	PAM8403 (3W)	~0.50€
Kaiutin	8Ω 2W, 50mm	~1–2€
Akut	2× 18650 Li-ion	~4–6€
Akkukotelo	2× 18650 rinnankytketty	~1€
Latauspiiri	TP4056 + DW01 suojapiiri	~0.50€
USB-C liitäntä	USB-C breakout board	~0.30€
Virtakytkin	Liukukytkin	~0.20€
Vastukset	10kΩ × 8, 1kΩ × 1, 5.1kΩ × 2, 4kΩ × 1	~0.20€
Yhteensä		**~10–13€**

Kytkennät

Napit → ESP32

Nappi	ESP32 GPIO	Pull-down
Vihreä fret	GPIO 13	10kΩ → GND

Nappi	ESP32 GPIO	Pull-down
Punainen fret	GPIO 12	10kΩ → GND
Keltainen fret	GPIO 14	10kΩ → GND
Sininen fret	GPIO 27	10kΩ → GND
Oranssi fret	GPIO 26	10kΩ → GND
Strum alas	GPIO 32	10kΩ → GND
Strum ylös	GPIO 33	10kΩ → GND
Select	GPIO 15	10kΩ → GND
Start	GPIO 16	10kΩ → GND
D-pad (kaikki suunnat)	GPIO 17	10kΩ → GND

Jokainen nappi: GPIO → nappi → 3.3V ja GPIO → 10kΩ → GND.
D-padin kaikki neljä suuntaa kytketään samaan GPIO 17 -pinniin.

Whammy bar → ESP32

```
Potentiometri keskipiste  →  GPIO 34 (ADC)
Potentiometri pää 1       →  3.3V
Potentiometri pää 2       →  GND
```

Mittaa whammy-lepotila-arvo ennen käyttöä: `Serial.println(mozziAnalogRead(34))` ja päivitä arvo koodiin.

Äänilähtö → PAM8403 → kaiutin

```
ESP32 GPIO25  →  1kΩ   →  PAM8403 L_IN+
ESP32 GND     →  10kΩ  →  GPIO25 (bias)
ESP32 GND             →  PAM8403 L_IN-
PAM8403 OUT+          →  Kaiutin +
PAM8403 OUT-          →  Kaiutin -
PAM8403 VCC           →  Virtakytkin OUT+
PAM8403 GND           →  GND
```

Akut, lataus ja virtakytkin

```
USB-C CC1   →  5.1kΩ  →  GND       (pakollinen USB-C tunnistus)
USB-C CC2   →  5.1kΩ  →  GND
USB-C VBUS  →  TP4056 IN+
```

```
USB-C GND   →  TP4056 IN-

Akku 1 (+)  ─┬─  TP4056 BAT+
Akku 2 (+)  ─┘
Akku 1 (-)  ─┬─  TP4056 BAT-
Akku 2 (-)  ─┘

TP4056 OUT+  →  Virtakytkin  →  ESP32 VIN
                             →  PAM8403 VCC
TP4056 OUT-  →  GND
```

TP4056 PROG-pinniin 4kΩ vastus → latausvirta 500mA.

Kesto-arvio: 2× 18650 ≈ 6000mAh, kulutus ~350mA → noin 17 tuntia yhdellä latauksella.

Sijoittelu kitaran sisään

```
[Kaula]
  ├── Vihreä fret   →  johdin  →  GPIO 13
  ├── Punainen      →  johdin  →  GPIO 12
  ├── Keltainen     →  johdin  →  GPIO 14
  ├── Sininen       →  johdin  →  GPIO 27
  └── Oranssi       →  johdin  →  GPIO 26

[Strum + muut napit — strummilevyn alla]
  ├── Strum alas    →  GPIO 32
  ├── Strum ylös    →  GPIO 33
  ├── Select        →  GPIO 15
  ├── Start         →  GPIO 16
  └── D-pad         →  GPIO 17 (kaikki suunnat samaan pinniin)

[Whammy — strummilevyn alla]
  └── Potentiometri →  GPIO 34

[Runko sisällä]
  ├── ESP32 DevKit
  ├── PAM8403 vahvistin
  ├── TP4056 + DW01 latauspiiri
  └── 2× 18650 akkukotelo

[Runko ulkopuolella]
  ├── Virtakytkin   (sivu tai strummilevyn viereinen)
  ├── USB-C portti  (sivu, lataus)
  └── Kaiutin 50mm  (pohja tai sivu)
```

Rakennusvaiheet

Vaihe 1 — Testaa koekytkennällä Ennen kitaran avaamista kytke ESP32 + PAM8403 + kaiutin koekytkentälevylle ja simuloi nappeja hyppyjohtimilla. Varmista että ääni toimii ja Serial Monitor näyttää oikeat viestit.

Vaihe 2 — Avaa kitara Ruuvit löytyvät kitaran pohjasta (T10 Torx). Irrota kaula varoen — johtimet kulkevat kaulan läpi runkoon.

Vaihe 3 — Kartoita johtimet Käytä yleismittaria (jatkuvuusmittaus) selvittääksesi kumpi johto on signaali ja kumpi maa jokaisessa napissa. Merkitse johtimet teipillä.

Vaihe 4 — Juota johdot Juota fret-nappien signaalijohtimet ESP32:n GPIO-pinneihin. Lisää 10kΩ pull-down vastus jokaisen GPIO:n ja GND:n väliin.

Vaihe 5 — Kytke whammy Whammy-potentiometri löytyy strummilevyn alta. Kolme johtoa: 3.3V, GPIO 34 ja GND.

Vaihe 6 — Latauspiiri Juota TP4056-moduuliin USB-C breakout board ja 5.1kΩ CC-vastukset. Kytke akut rinnan BAT-pinneihin. Asenna PROG-pinniin 4kΩ vastus.

Vaihe 7 — Asenna komponentit runkoon ESP32, PAM8403 ja TP4056 kiinnitetään kitaran rungon sisälle kaksipuoleisella teipillä tai pienillä nippusiteillä. Akkukotelo sopii rungon alaosaan.

Vaihe 8 — Virtakytkin Asenna liukukytkin kitaran sivuun tai strummilevyn viereen. Kytke TP4056 OUT+:n ja ESP32 VIN:n väliin.

Vaihe 9 — Kaiuttimen reikä Poraa tai leikkaa 50mm reikä kitaran rungon pohjaan tai sivuun. Kiinnitä kaiutin reiän päälle ruuveilla tai liimalla.

Vaihe 10 — USB-C latausportti Tee pieni suorakaiteen muotoinen reikä kitaran sivuun USB-C breakout boardia varten. Kiinnitä epoksiliimalla.

Vaihe 11 — Mittaa whammy-lepotila Lataa koodi, avaa Serial Monitor ja aja `Serial.println(mozziAnalogRead(34))`. Kirjaa arvo kun whammy on lepotilassa ja päivitä `2048` koodissa oikeaksi arvoksi.

Vaihe 12 — Viimeistely Sulje kitara, kiristä ruuvit ja testaa kaikki napit. Lataa akku täyteen USB-C:llä ennen ensimmäistä käyttöä.

Vianetsintä

Ongelma	Syy	Ratkaisu
Ei ääntä	PAM8403 ei saa virtaa	Tarkista virtakytkin ja VCC-johto
Ääni säröilee	Liian kova volyymi	Lisää sarjavastus L_IN+-johtoon (2.2kΩ)
Nappi ei reagoi	Pull-down puuttuu	Lisää 10kΩ GPIO → GND
Whammy ei toimi	Väärä lepotila-arvo	Mittaa arvo ja päivitä koodiin
ESP32 ei käynnisty	Akku liian tyhjä	Lataa USB-C:llä ennen käyttöä
Mozzi ei käänny	Kirjasto puuttuu	Asenna Mozzi Library Managerista

---

Lähdekoodi

Tallenna tiedostoksi `gh_synth.ino` ja avaa Arduino IDE:ssä.

Vaatimukset:

- Arduino IDE 2.x
- ESP32 board package (Boards Manager: `esp32 by Espressif`)
- Kirjasto: `Mozzi by Tim Barrass` (Library Manager)

```
#include <MozziGuts.h>
#include <Oscil.h>
#include <ADSR.h>
#include <tables/sin2048_int8.h>

#define CONTROL_RATE 128
#define NUM_OSC 15

#define PIN_GREEN   13
#define PIN_RED     12
#define PIN_YELLOW  14
#define PIN_BLUE    27
#define PIN_ORANGE  26
#define PIN_STRUM_D 32
#define PIN_STRUM_U 33
#define PIN_SELECT  15
#define PIN_START   16
#define PIN_DPAD    17
#define PIN_WHAMMY  34

const int FRET_PINS[5] = {
  PIN_GREEN, PIN_RED, PIN_YELLOW, PIN_BLUE, PIN_ORANGE
};

const float CHORDS[3][5][3] = {
  {
    { 196.00, 246.94, 293.66 },
    { 130.81, 164.81, 196.00 },
    { 146.83, 185.00, 220.00 },
    { 220.00, 261.63, 329.63 },
    { 164.81, 196.00, 246.94 },
  },
  {
    { 164.81, 207.65, 246.94 },
    { 110.00, 138.59, 164.81 },
    { 146.83, 185.00, 220.00 },
    { 196.00, 246.94, 293.66 },
    { 123.47, 155.56, 185.00 },
  },
  {
    { 130.81, 164.81, 196.00 },
    { 196.00, 246.94, 293.66 },
    { 220.00, 261.63, 329.63 },
    { 174.61, 220.00, 261.63 },
    { 164.81, 196.00, 246.94 },
  },
```

```
};

const float NOTES[3][5] = {
  {  65.41,  73.42,  82.41,  98.00, 110.00 },
  { 130.81, 146.83, 164.81, 196.00, 220.00 },
  { 261.63, 293.66, 329.63, 392.00, 440.00 },
};

const char* SET_NAMES[3]  = { "Pop", "Rock", "Balladi" };
const char* OCT_NAMES[3]  = { "Basso", "Normaali", "Melodia" };
const char* CHORD_NAMES[3][5] = {
  { "G", "C", "D", "Am", "Em" },
  { "E", "A", "D", "G",  "B"  },
  { "C", "G", "Am","F",  "Em" },
};

bool chordMode    = true;
int  currentSet   = 0;
int  currentOct   = 1;
bool vibratoOn    = false;
int  vibratoPhase = 0;

Oscil<SIN2048_NUM_CELLS, AUDIO_RATE> osc[NUM_OSC] = {
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
  Oscil<SIN2048_NUM_CELLS, AUDIO_RATE>(SIN2048_DATA),
};
ADSR<CONTROL_RATE, AUDIO_RATE> env[NUM_OSC];

bool fretHeld[5]   = {false};
bool strumDownLast = false;
bool strumUpLast   = false;
bool selectLast    = false;
bool startLast     = false;
bool dpadLast      = false;
```

```
void triggerChord(int fret) {
  for (int n = 0; n < 3; n++) {
    int idx = fret * 3 + n;
    osc[idx].setFreq(CHORDS[currentSet][fret][n]);
    env[idx].noteOn();
  }
  Serial.println("Sointu: " + String(CHORD_NAMES[currentSet][fret]));
}

void releaseChord(int fret) {
  for (int n = 0; n < 3; n++)
    env[fret * 3 + n].noteOff();
}

void triggerNote(int fret) {
  int idx = fret * 3;
  osc[idx].setFreq(NOTES[currentOct][fret]);
  env[idx].noteOn();
  Serial.print("Nuotti: ");
  Serial.print(NOTES[currentOct][fret]);
  Serial.println(" Hz");
}

void releaseNote(int fret) {
  env[fret * 3].noteOff();
}

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 5; i++)
    pinMode(FRET_PINS[i], INPUT);
  pinMode(PIN_STRUM_D, INPUT);
  pinMode(PIN_STRUM_U, INPUT);
  pinMode(PIN_SELECT,  INPUT);
  pinMode(PIN_START,   INPUT);
  pinMode(PIN_DPAD,    INPUT);
  for (int i = 0; i < NUM_OSC; i++) {
    env[i].setADLevels(255, 180);
    env[i].setTimes(5, 80, 150, 400);
  }
  startMozzi(CONTROL_RATE);
  Serial.println("Kaynnistetty - Sointutila / Pop");
}

void updateControl() {
  for (int i = 0; i < 5; i++) {
```

```
  bool pressed = digitalRead(FRET_PINS[i]) == HIGH;
  if (!pressed && fretHeld[i])
    chordMode ? releaseChord(i) : releaseNote(i);
  fretHeld[i] = pressed;
}

bool sd = digitalRead(PIN_STRUM_D) == HIGH;
bool su = digitalRead(PIN_STRUM_U) == HIGH;
if ((sd && !strumDownLast) || (su && !strumUpLast)) {
  bool any = false;
  for (int i = 0; i < 5; i++) {
    if (fretHeld[i]) {
      chordMode ? triggerChord(i) : triggerNote(i);
      any = true;
    }
  }
  if (!any) chordMode ? triggerChord(0) : triggerNote(0);
}
strumDownLast = sd;
strumUpLast   = su;

bool sel = digitalRead(PIN_SELECT) == HIGH;
if (sel && !selectLast) {
  chordMode = !chordMode;
  Serial.println(chordMode ? "Tila: Sointutila" : "Tila: Nuottitila");
}
selectLast = sel;

bool sta = digitalRead(PIN_START) == HIGH;
if (sta && !startLast) {
  if (chordMode) {
    currentSet = (currentSet + 1) % 3;
    Serial.println("Setti: " + String(SET_NAMES[currentSet]));
  } else {
    currentOct = (currentOct + 1) % 3;
    Serial.println("Oktaavi: " + String(OCT_NAMES[currentOct]));
  }
}
startLast = sta;

bool dp = digitalRead(PIN_DPAD) == HIGH;
if (dp && !dpadLast) {
  vibratoOn = !vibratoOn;
  Serial.println(vibratoOn ? "Vibrato: ON" : "Vibrato: OFF");
}
dpadLast = dp;
```

```
  int whammy = mozziAnalogRead(PIN_WHAMMY);
  float bend = (float)(whammy - 2048) / 2048.0f * 0.05f;

  float vibrato = 0;
  if (vibratoOn) {
    vibratoPhase = (vibratoPhase + 1) % CONTROL_RATE;
    vibrato = 0.015f * sin(2.0f * 3.14159f * vibratoPhase
              / (float)CONTROL_RATE * 5);
  }

  for (int i = 0; i < 5; i++) {
    for (int n = 0; n < 3; n++) {
      int idx = i * 3 + n;
      float base = chordMode
        ? CHORDS[currentSet][i][n]
        : (n == 0 ? NOTES[currentOct][i] : 0);
      if (base > 0)
        osc[idx].setFreq(base * (1.0f + bend + vibrato));
      env[idx].update();
    }
  }
}

AudioOutput_t updateAudio() {
  int32_t out = 0;
  for (int i = 0; i < NUM_OSC; i++)
    out += (int32_t)env[i].next() * osc[i].next() >> 8;
  return MonoOutput::fromAlmostNBit(11, out / 5);
}

void loop() { audioHook(); }
```